

Moving 40 TB of historical data from an RDBMS to a modern data storage solution like Iceberg on Google Cloud Storage (GCS) with Dataproc can be an effective approach to optimize costs and improve scalability. Here's a step-by-step plan to achieve this:

1. Why Iceberg on GCS?

Iceberg is an open table format that:

- Supports large-scale analytic workloads.
- Provides schema evolution and time travel capabilities.
- Stores data efficiently in columnar formats (e.g., Parquet).
- Works seamlessly with big data tools like Apache Spark on Dataproc.

GCS offers low-cost, scalable, and durable storage, significantly reducing storage costs compared to traditional RDBMS.

2. Cost-Saving Features

- **Decoupled Storage and Compute:** Store data on GCS and process it using Dataproc only when needed, minimizing costs.
 - **Efficient Data Access:** Iceberg's columnar format and metadata pruning reduce compute costs by avoiding unnecessary scans.
 - **Lifecycle Management:** Use GCS bucket policies for automatic archival or deletion of older data.
-

3. Migration Plan

Step 1: Analyze and Export Historical Data

- **Assess the Schema:** Identify the RDBMS schema, tables, and columns. Plan the mapping to Iceberg tables.
- **Partition Strategy:** Define partitions (e.g., date-based or region-based) to optimize query performance in Iceberg.
- **Export Data:** Export RDBMS data in a structured format like CSV, JSON, or Parquet using database tools or custom scripts.

Step 2: Set Up GCS and Iceberg Table Structure

- **GCS Bucket Setup:** Create a GCS bucket and set lifecycle policies for cost management.
- **Iceberg Table Setup:** Use Dataproc with Apache Spark to create Iceberg tables, specifying schema and partitioning.

Step 3: Load Data into Iceberg

- **Dataproц Cluster:** Spin up a Dataproц cluster with the necessary configurations for Iceberg and GCS.
- **Data Transformation:** Use Apache Spark on Dataproц to transform exported RDBMS data (if needed) into Parquet or Avro format.
- **Data Loading:** Write transformed data into Iceberg tables on GCS using Spark.

Example Spark Code for Writing Data to Iceberg:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("RDBMS to Iceberg Migration") \
    .config("spark.sql.catalog.spark_catalog", "org.apache.iceberg.spark.SparkCatalog") \
    .config("spark.sql.catalog.spark_catalog.type", "hadoop") \
    .config("spark.sql.catalog.spark_catalog.warehouse", "gs://your-bucket-name/warehouse") \
    .getOrCreate()
```

```
# Read exported data
```

```
df = spark.read.parquet("gs://your-bucket-name/exported-data/")
```

```
# Write to Iceberg
```

```
df.write.format("iceberg").mode("append").save("spark_catalog.db_name.table_name")
```

Step 4: Optimize and Query Data

- **Compaction:** Periodically compact small files into larger ones to reduce storage costs and improve performance.
 - **Queries:** Use Dataproц or BigQuery (via Iceberg integration) for efficient querying.
-

4. Cost Analysis

a. GCS Costs

- Storage: ~\$0.02/GB/month for standard storage.
- Lifecycle Policies: Archive older data to Coldline or Archive storage (~\$0.004/GB/month).

b. Dataproц Costs

- Pay-per-use: Dataproc charges for the compute time during data processing only.
- Use preemptible VMs for batch jobs to reduce costs by ~70-80%.

c. RDBMS Cost Savings

- Avoid high licensing and infrastructure costs associated with traditional RDBMS systems for historical data.

5. Benefits

1. **Scalability:** GCS and Iceberg handle massive datasets effortlessly.
2. **Cost Efficiency:** Pay only for storage and occasional compute.
3. **Flexibility:** Analyze data using various tools (Dataproc, BigQuery, Spark).
4. **Durability:** GCS ensures data redundancy and reliability.

This approach combines Iceberg's powerful table format with GCS's cost-efficient storage and Dataproc's scalable compute, significantly reducing costs while ensuring high performance and flexibility.

Follow [Ahmed Mohiuddin | LinkedIn](#)